

## Backpropagation

### Overview

- apply the backpropagation algorithm for training MLPs
- utilize Autograd systems for automatic differentiation
- implement strategies for effective neural network initialization, regularization, and learning rate adaptation

### Async

### Backpropagation and the Chain Rule

#### The Chain Rule of Differentiation

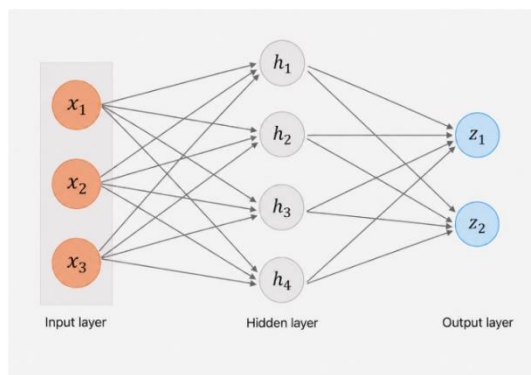
- common version

$$\frac{dz(y)}{dx} = \frac{dz}{dy} \cdot \frac{dy}{dx}$$

- less-common version

$$\text{suppose } z = z(y_1(x), y_2(x), \dots, y_n(x))$$

$$\frac{dz(y)}{dx} = \sum_{i=1}^N \frac{dz}{dy_i} \cdot \frac{dy_i}{dx}$$



## Math Derivation for Backpropagation

- terminology
  - $L$ : # of layers
  - $N^l$ : dimensionality of layer  $l$
  - $W^l$ : weight of matrix for layer  $l$
  - $b^l$ : bias vector for layer  $l$
  - $z^l$ : linear pre-activations for layer  $l$ 
    - $(z^l = W^l a^{l-1} + b^l)$
  - $\sigma^l$ : activation function for layer  $l$
  - $a^l$ : nonlinear activations for layer  $l$ 
    - $(a^l = \sigma^l(z^l))$
  - $a^0$  and  $a^L$ : refer to the model inputs ( $x$ ) and model predictions ( $\hat{y}(x)$ ), respectively
- backpropagation preliminaries
  - for training NNs with gradient descent, we want  $\frac{dL}{dW^l}$
  - we can compute this if we have  $\frac{dL}{da^L}$  and  $\frac{da^l}{dz^l}$

derivative  
of loss

derivative of  
nonlinear activations

- using the chain rule, we have:

$$\frac{dL}{dW_{ij}^l} = \sum_{k=1}^{N^l} \frac{dL}{dz_k^l} \frac{dz_k^l}{dW_{ij}^l}$$

- in matrix notation:

$$\frac{dL}{dW^l} = \frac{dL}{dz^l} a^{l-1T}$$

- we can calculate these iteratively, starting from  $l = L$  and going down to  $l = 1$

- when:  $l = L$ :

$$\frac{dL}{dz_k^L} = \frac{dL}{da_k^L} \cdot \frac{da_k^L}{dz_k^L}$$

- for:  $l = L - 1, L - 2, \dots, 1$ :

$$\frac{d\mathcal{L}}{dz_k^l} = \frac{d\mathcal{L}}{da_k^l} \cdot \frac{da_k^l}{dz_k^l} = \left\{ \sum_{m=1}^{N^{l+1}} \frac{d\mathcal{L}}{dz_m^{l+1}} \cdot \frac{dz_m^{l+1}}{da_k^l} \right\} \cdot \frac{da_k^l}{dz_k^l}$$

- so:

$$\frac{d\mathcal{L}}{dz_k^l} = \left\{ \sum_{m=1}^{N^{l+1}} \frac{d\mathcal{L}}{dz_m^{l+1}} \cdot W_{mk}^{l+1} \right\} \cdot \frac{da_k^l}{dz_k^l}$$

- compact notation:

$$\frac{d\mathcal{L}}{dz^l} = \left( W^{(l+1)^T} \frac{dL}{dz^{l+1}} \right) \circ \frac{da^l}{dz^l}$$

element-wise  
multiplication

### Procedure for Backpropagation

- the goal of backpropagation is to calculate the derivative of the loss function with respect to model parameters  $dW$  for any weight  $W$
- how to compute  $\frac{dL}{dW}$  for any weight  $W$ :

- 1) compute  $\frac{dL}{dz_k^L} = \frac{dL}{da_k^L} \cdot \frac{da_k^L}{dz_k^L}$

- 2) iteratively compute  $\frac{dL}{dz^l}$  for  $l = L - 1, L - 2, \dots, 3, 2, 1$

- a.  $\frac{d\mathcal{L}}{dz^l} = \left( W^{(l+1)^T} \frac{dL}{dz^{l+1}} \right) \circ \frac{da^l}{dz^l}$

- 3) from (2), compute  $\frac{dL}{dW^l} = \frac{dL}{dz^l} \cdot a^{l-1^T}$

**Backpropagation****Example**

- consider a FFT for a binary classification task

- assume **sigmoid** (like LR)

$$a^l = (1 + \exp(-z^l))^{-1} \forall l \in \{1 \dots L\}$$

in the **prediction** and **intermediate** layers

- compute  $\frac{dL}{dz_k^L} = \frac{dL}{da_k^L} \cdot \frac{da_k^L}{dz_k^L}$

$$a^L(1 - a^L)$$

derivative of sigmoid activation

○ you are left with

$$a^L - y$$

$$\frac{a^L - y}{a^L(1 - a^L)}$$

derivative of binary cross-entropy (CE) loss

- iteratively compute  $\frac{dL}{dz^l}$  for  $l = L - 1, L - 2, \dots, 3, 2, 1$

$$\frac{d\mathcal{L}}{dz^l} = \left( W^{(l+1)T} \frac{dL}{dz^{l+1}} \right) \circ \frac{da^l}{dz^l} \rightarrow$$

$$\frac{d\mathcal{L}}{dz^{L-1}} = W^{LT(a^L - y)} \circ a^{L-1}(1 - a^{L-1}) \rightarrow$$

$$\frac{d\mathcal{L}}{dz^{L-2}} = W^{LT(a^L - y)} \circ a^{L-1}(1 - a^{L-1})a^{L-2T} \rightarrow$$

and so on ...

**Vanishing and  
Exploding  
Gradients**

- in the backpropagation step, we have

$$\frac{d\mathcal{L}}{dz^l} = \left( W^{(l+1)T} \frac{dL}{dz^{l+1}} \right) \circ \frac{da_k^l}{dz_k^l}$$

L                      R

- for networks with many layers, the weight matrices computed iteratively to the left and the activation derivative computed to the right, can cause gradients to blow up, or shrink to approaching zero

